

Operations: Logs, Metrics, Traces, Alerts, Rate Limiting, Security Basics

1. Overview

Operations is the part of system design that keeps a real system observable, reliable, safe, and controlled after it is deployed.

In interviews, "operations" usually means:

- **Logs:** detailed event records produced by applications and infrastructure.
- **Metrics:** numeric measurements over time, such as latency, error rate, CPU usage, or request count.
- **Traces:** end-to-end request journeys across services.
- **Alerts:** notifications triggered when something important may be broken.
- **Rate limiting:** controlling how many requests a user, client, or service can make.
- **Security basics:** protecting data, users, services, and infrastructure from misuse or attack.

Definition

Operations in system design means designing the runtime support around an application so engineers can monitor it, debug it, protect it, and keep it reliable under real-world traffic.

Why It Matters

A system is not complete just because the code works locally. Real systems face:

- Traffic spikes
- Slow dependencies
- Bugs in production
- Abuse and spam
- Data leaks
- Hardware failures
- Bad deployments
- Partial outages

Operations helps answer questions like:

- Is the system healthy?
- Why is latency high?
- Which service is failing?
- Are users getting errors?
- Is someone abusing the API?
- Are we protecting sensitive data?

Where It Is Used In Real Systems

Operations appears in almost every production system:

- Backend APIs
- Microservices
- Databases
- Payment systems
- Search systems
- Messaging platforms
- Cloud infrastructure
- Mobile and web applications
- Distributed systems

Example: In a food delivery app, operations helps detect that order placement is failing in one city, trace the issue to the payment service, alert the on-call engineer, and rate-limit repeated retry traffic.

Why Interviewers Ask About It

Interviewers ask about operations because strong SDEs do not only design happy-path architecture. They also think about:

- Production debugging
- Reliability
- Failure handling
- Abuse prevention
- Monitoring
- Security
- Scalability under real traffic

A candidate who discusses logs, metrics, traces, alerts, rate limits, and security shows production maturity.

2. Core Idea

The core idea is: **a production system must be observable, controllable, and secure.**

Intuition

Imagine a large building:

- Logs are like security camera footage.
- Metrics are like dashboard gauges for electricity, water, and temperature.
- Traces are like tracking one visitor's path through the building.
- Alerts are like fire alarms.
- Rate limiting is like entry control at the gate.
- Security is like locks, ID cards, guards, and access rules.

Without these, the building may work on a normal day, but nobody knows what is happening during an emergency.

Real-World Analogy

Think of a hospital:

System Concept	Hospital Analogy
Logs	Doctor notes and event records
Metrics	Heart rate, blood pressure, oxygen level
Traces	Patient journey from emergency room to lab to surgery
Alerts	Alarm when oxygen level drops
Rate limiting	Limiting visitor entry or appointment slots
Security	Patient privacy, ID checks, restricted rooms

The hospital does not just treat patients. It continuously measures, records, alerts, and protects.

Small Example

Suppose an API endpoint `POST /orders` is slow.

Good operations design gives you:

- Logs: "Payment service timeout for order_id=123"
- Metrics: p95 latency increased from 200 ms to 2 seconds
- Trace: request spent 1.7 seconds inside payment service
- Alert: checkout error rate is above threshold
- Rate limit: one user cannot spam 1000 order requests per minute
- Security: user can only create orders for their own account

Step-By-Step Explanation

1. A user sends a request to your backend.
2. The backend emits logs for important events.
3. The backend records metrics such as request count, latency, and errors.
4. A trace ID follows the request across multiple services.
5. Monitoring checks if metrics violate expected limits.
6. Alerts notify engineers when user impact is likely.
7. Rate limiting blocks abusive or accidental high request volume.
8. Security controls verify identity, permission, input safety, and data protection.

3. Important Subtopics

3.1 Logs

What It Means

Logs are timestamped records of events generated by applications, services, databases, or infrastructure.

Example log:

```
2026-07-09T10:15:30Z level=ERROR service=order-api trace_id=abc123 user_id=42
```

```
msg="Payment timeout" payment_provider=stripe duration_ms=3000
```

Why It Matters

Logs help debug specific incidents:

- What happened?
- When did it happen?
- Which user/request was affected?
- Which service produced the error?
- What context explains the failure?

Example

If login fails for a user, logs may show:

```
level=WARN event=login_failed reason=invalid_password user_id=101 ip=10.2.3.4
```

Common Interview Angle

Interviewers may ask:

- What should you log?
- What should you avoid logging?
- How do logs help in distributed systems?

Strong answer: log important events with structured fields, correlation IDs, error context, and avoid sensitive data like passwords or tokens.

3.2 Structured Logging

What It Means

Structured logs use machine-readable key-value or JSON format instead of plain text.

Example:

```
{
  "level": "ERROR",
  "service": "payment-service",
  "trace_id": "abc123",
  "order_id": "ord_99",
  "error": "gateway_timeout"
}
```

Why It Matters

Structured logs are easier to search, filter, aggregate, and alert on.

Example

You can query:

```
service="payment-service" AND error="gateway_timeout"
```

Common Interview Angle

Plain text logs are readable but harder to process at scale. Structured logs are preferred in production systems.

3.3 Metrics

What It Means

Metrics are numeric measurements collected over time.

Common metrics:

- Request rate
- Error rate
- Latency
- CPU usage
- Memory usage
- Queue length
- Database connections
- Cache hit ratio

Why It Matters

Metrics give a high-level health view of the system.

Example

```
http_requests_total{service="order-api", status="500"} 120  
http_request_duration_ms_p95{service="order-api"} 850
```

Common Interview Angle

Interviewers expect candidates to mention the "golden signals":

Golden Signal	Meaning
Latency	How long requests take

Golden Signal	Meaning
Traffic	How many requests are served
Errors	How many requests fail
Saturation	How full or overloaded resources are

3.4 Percentiles And Latency

What It Means

Latency percentiles show how slow requests are for different portions of users.

Metric	Meaning
p50	50% of requests are faster than this
p90	90% of requests are faster than this
p95	95% of requests are faster than this
p99	99% of requests are faster than this

Why It Matters

Averages hide bad user experiences.

Example:

```
Average latency = 100 ms  
p99 latency = 3000 ms
```

This means most users are fine, but the slowest 1% have a terrible experience.

Common Interview Angle

Do not only mention average latency. Production systems track p95 and p99.

3.5 Traces

What It Means

A trace records the path of one request across services. Each operation in the trace is called a span.

Why It Matters

In microservices, one user request may touch many services:

```
Client -> API Gateway -> Order Service -> Payment Service -> Inventory Service -> Notification Service
```

Tracing shows where time was spent and where failure occurred.

Example

```
trace_id=abc123

API Gateway      20 ms
Order Service    80 ms
Payment Service  1200 ms
Inventory Service 50 ms
```

The trace clearly shows payment is the bottleneck.

Common Interview Angle

Interviewers may ask how to debug latency in a distributed system. Mention distributed tracing with trace IDs propagated across services.

3.6 Correlation ID / Request ID / Trace ID

What It Means

A correlation ID is a unique ID attached to a request so all logs, metrics, and traces related to that request can be connected.

Why It Matters

Without correlation IDs, debugging across services is painful.

Example

```
trace_id=abc123 appears in:
order-api logs
payment-service logs
inventory-service logs
```

Common Interview Angle

For distributed systems, always mention passing trace IDs through headers, such as:

X-Request-ID: abc123

3.7 Alerts

What It Means

Alerts notify engineers when the system may need human attention.

Why It Matters

Alerts help reduce downtime and user impact.

Example

Trigger alert when:

checkout_error_rate > 5% for 5 minutes

Common Interview Angle

Good alerts should be actionable. Avoid alerting on every small internal fluctuation.

3.8 SLI, SLO, SLA

What It Means

Term	Meaning
SLI	Service Level Indicator: what you measure
SLO	Service Level Objective: target you aim for
SLA	Service Level Agreement: customer contract

Why It Matters

These help define reliability expectations.

Example

Term	Example
SLI	Request success rate
SLO	99.9% successful requests per month
SLA	Refund if uptime drops below 99.5%

Common Interview Angle

Interviewers like candidates who can distinguish internal targets from external contracts.

3.9 Rate Limiting

What It Means

Rate limiting restricts how many requests a client can make in a time window.

Why It Matters

It protects systems from:

- Abuse
- Bots
- Scraping
- Denial-of-service attempts
- Accidental client bugs
- Overloaded downstream services

Example

```
Allow 100 requests per user per minute.
```

If user `u1` sends 150 requests in one minute, the last 50 are rejected or delayed.

Common Interview Angle

Interviewers may ask how to design rate limiting at scale. Mention algorithms like token bucket, leaky bucket, fixed window, and sliding window.

3.10 Rate Limiting Algorithms

Fixed Window Counter

Counts requests in fixed intervals.

```
10:00:00 - 10:00:59 => max 100 requests  
10:01:00 - 10:01:59 => max 100 requests
```

Simple, but allows bursts at window boundaries.

Sliding Window Log

Stores timestamps of recent requests and removes old ones.

Accurate but memory-heavy.

Sliding Window Counter

Approximates sliding behavior using current and previous windows.

Better than fixed window and less expensive than sliding log.

Token Bucket

Tokens refill at a fixed rate. Each request consumes one token.

Allows controlled bursts.

Leaky Bucket

Requests are processed at a steady rate, like water leaking from a bucket.

Smooths bursts.

3.11 Security Basics

What It Means

Security basics are the minimum protections every system should have.

Core areas:

- Authentication
- Authorization
- Input validation
- Encryption
- Secrets management
- Secure logging
- Network security
- Principle of least privilege

Why It Matters

A scalable system that leaks user data or allows unauthorized access is a failed system.

Example

In an order service:

- Authentication verifies who the user is.
- Authorization checks whether the user can view the order.
- Input validation prevents invalid or malicious input.
- Encryption protects data in transit and at rest.
- Logs avoid storing credit card numbers.

Common Interview Angle

Interviewers do not expect every candidate to be a security expert, but they expect safe basics.

3.12 Authentication Vs Authorization

What It Means

Concept	Meaning
Authentication	Verifies identity
Authorization	Verifies permission

Why It Matters

Confusing these is a common interview mistake.

Example

Authentication:

```
This is Alice.
```

Authorization:

```
Alice can view order #123.
```

Common Interview Angle

Always mention both. A logged-in user should not automatically access every resource.

4. Real-World Example

Example: Designing Operations For An E-Commerce Checkout System

Suppose you are designing checkout for an e-commerce platform.

Basic flow:

```
User -> API Gateway -> Cart Service -> Order Service -> Payment Service ->  
Inventory Service -> Notification Service
```

Logs

Each service logs important events:

```
order_created
payment_authorized
inventory_reserved
email_sent
payment_failed
```

Each log includes:

- timestamp
- service
- level
- trace_id
- user_id
- order_id
- error details when relevant

Metrics

Important metrics:

Metric	Why It Matters
Checkout success rate	Measures business-critical health
Payment failure rate	Detects payment issues
p95 checkout latency	Measures user experience
Inventory reservation failures	Detects stock service problems
Queue size	Detects backlog
5xx error rate	Detects server failures

Traces

One checkout request might produce this trace:

```
Trace ID: tr_789

API Gateway      15 ms
Cart Service     40 ms
Order Service    90 ms
Payment Service  1800 ms
Inventory Service 60 ms
Notification     20 ms
```

This shows payment is slow.

Alerts

Useful alerts:

```
checkout_success_rate < 98% for 5 minutes
payment_error_rate > 3% for 5 minutes
p95_checkout_latency > 2 seconds for 10 minutes
queue_depth > 10000 for 15 minutes
```

Rate Limiting

Possible rules:

```
Login: 5 attempts per minute per IP
Checkout: 20 attempts per user per minute
Coupon API: 50 requests per user per hour
Search API: 100 requests per minute per IP
```

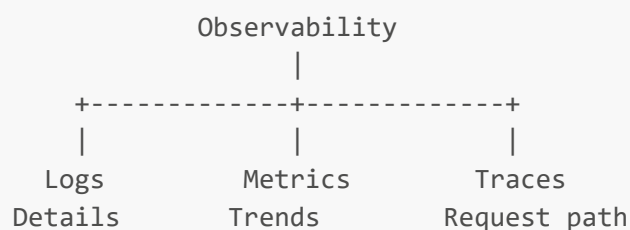
Security

Security basics:

- Use HTTPS.
- Hash passwords with strong password hashing algorithms.
- Never log passwords, tokens, or full card numbers.
- Use authorization checks for every user-owned resource.
- Store secrets in a secret manager.
- Validate input for all public APIs.
- Use idempotency keys for payment requests.

5. Diagrams / Mental Models

Observability Triangle



Request With Operational Signals

```

Client
|
| request_id=abc123
v
API Gateway ---- logs request started
|
v
Order Service ---- emits latency metric
|
v
Payment Service ---- trace shows slow span
|
v
Database ---- emits connection and query metrics
|
v
Response

```

Alert Flow

```

Application emits metrics
|
v
Monitoring system evaluates rules
|
v
Threshold breached for enough time
|
v
Alert sent to on-call engineer
|
v
Engineer checks dashboard, logs, traces

```

Rate Limiter Flow

```

Incoming Request
|
v
Identify key: user_id / IP / API key
|
v
Check request count or token availability
|
+-----+-----+
|         |         |
Allowed   Limited
|         |         |

```

```
      v      v
Process  Return 429 Too Many Requests
```

Security Request Flow

```
Request
|
v
TLS protects data in transit
|
v
Authenticate user
|
v
Authorize action
|
v
Validate input
|
v
Process request
|
v
Log safely without sensitive data
```

6. Common Interview Questions

1. What is the difference between logs, metrics, and traces?

Answer: Logs are detailed event records, metrics are numeric time-series measurements, and traces show the journey of a request across services.

Key points interviewer expects:

- Logs answer "what happened?"
- Metrics answer "how is the system behaving overall?"
- Traces answer "where did this request spend time or fail?"

Common mistakes:

- Saying logs and metrics are the same.
- Ignoring traces in distributed systems.
- Only discussing debugging, not monitoring.

2. What should you log in a backend service?

Answer: Log important business and technical events, errors, request IDs, service names, relevant entity IDs, status codes, latency, and failure reasons.

Key points interviewer expects:

- Use structured logs.
- Include correlation IDs.
- Log errors with context.
- Avoid sensitive data.

Common mistakes:

- Logging everything blindly.
- Logging passwords, tokens, or full credit card numbers.
- Not including request IDs.

3. Why are metrics important?

Answer: Metrics show system health over time and help detect problems such as high latency, increased errors, traffic spikes, or resource saturation.

Key points interviewer expects:

- Metrics support dashboards and alerts.
- Metrics are cheaper to aggregate than raw logs.
- Important metrics include latency, traffic, errors, and saturation.

Common mistakes:

- Only mentioning CPU and memory.
- Ignoring user-facing metrics like success rate and p95 latency.

4. What are p95 and p99 latency?

Answer: p95 latency means 95% of requests are faster than that value. p99 means 99% of requests are faster than that value.

Key points interviewer expects:

- Percentiles show tail latency.
- Tail latency matters for user experience.
- Average latency can hide slow requests.

Common mistakes:

- Thinking p99 means 99% failure rate.
- Only reporting average latency.

5. How would you debug high latency in a microservices system?

Answer: Start with metrics to confirm latency increase, check dashboards for affected services, use distributed tracing to find slow spans, inspect logs with the same trace ID, and check downstream dependencies such as databases, caches, and external APIs.

Key points interviewer expects:

- Use metrics first for scope.
- Use traces to locate bottleneck.
- Use logs for detailed cause.
- Check recent deployments and dependency health.

Common mistakes:

- Jumping directly to code changes.
- Looking only at one service.
- Ignoring downstream systems.

6. What makes a good alert?

Answer: A good alert is actionable, user-impact focused, has a clear threshold, avoids noise, and includes enough context for diagnosis.

Key points interviewer expects:

- Alert on symptoms, not every internal cause.
- Use time windows to avoid flapping.
- Include runbook or dashboard links when possible.

Common mistakes:

- Alerting on every log error.
- Creating noisy alerts that engineers ignore.
- Alerting on non-actionable events.

7. What is rate limiting and why is it needed?

Answer: Rate limiting controls how many requests a client can make in a time period. It protects the system from abuse, bots, accidental client bugs, and overload.

Key points interviewer expects:

- Protects backend resources.
- Can be per IP, user, API key, route, or tenant.
- Often returns HTTP 429 Too Many Requests.

Common mistakes:

- Treating rate limiting only as security.
- Forgetting distributed rate limiting.
- Not defining the limiting key.

8. Compare token bucket and leaky bucket.

Answer: Token bucket allows bursts up to bucket capacity while maintaining an average refill rate. Leaky bucket processes requests at a fixed output rate and smooths bursts.

Key points interviewer expects:

- Token bucket is good for APIs that allow short bursts.
- Leaky bucket is good when steady processing is required.
- Both protect systems from overload.

Common mistakes:

- Saying they are identical.
- Forgetting burst behavior.

9. How do you implement rate limiting in a distributed system?

Answer: Use a shared store such as Redis to keep counters or token state across multiple application instances. Use atomic operations or Lua scripts to avoid race conditions.

Key points interviewer expects:

- Multiple servers need shared state.
- Redis is commonly used.
- Need expiry for window-based counters.
- Handle store failure carefully.

Common mistakes:

- Keeping counters only in local memory.
- Ignoring race conditions.
- Not setting TTLs.

10. What is the difference between authentication and authorization?

Answer: Authentication verifies who the user is. Authorization verifies what the user is allowed to do.

Key points interviewer expects:

- AuthN: identity
- AuthZ: permission
- Both are required for secure systems.

Common mistakes:

- Saying login alone is enough.
- Not checking resource ownership.

11. What security practices should every backend API follow?

Answer: Use HTTPS, authenticate users, authorize every protected action, validate input, avoid logging secrets, hash passwords securely, protect secrets, and apply least privilege.

Key points interviewer expects:

- Defense in depth.
- Secure password storage.
- Input validation.

- Sensitive data protection.

Common mistakes:

- Storing plain-text passwords.
- Putting secrets in code.
- Trusting client input.

12. Why should sensitive data not be logged?

Answer: Logs are often copied, indexed, retained, and accessed by many tools and people. Logging sensitive data can cause security incidents and privacy violations.

Key points interviewer expects:

- Avoid passwords, tokens, card numbers, and personal data.
- Mask or redact when needed.
- Logs should follow retention and access policies.

Common mistakes:

- Thinking logs are internal and therefore safe.
- Logging full request bodies blindly.

7. Deep-Dive Questions

1. How do you design observability for a new microservice?

Start with standard instrumentation:

- Structured logs with service name, environment, trace ID, request ID, and error context.
- Metrics for request count, error rate, latency percentiles, queue depth, resource usage, and dependency latency.
- Distributed traces across inbound requests, outbound calls, database calls, and message queue operations.
- Dashboards for service health.
- Alerts based on user impact and SLOs.

Also define log retention, sampling, sensitive data redaction, and ownership for alerts.

2. How would you reduce alert fatigue?

Use fewer, better alerts:

- Alert on user impact, not every internal symptom.
- Add time windows, such as "for 5 minutes."
- Group related alerts.
- Remove duplicate alerts.
- Add severity levels.
- Use runbooks.
- Review noisy alerts regularly.

Bad alerting makes engineers ignore real incidents.

3. How do you handle high-cardinality metrics?

High cardinality means too many unique label combinations, such as one metric series per user ID or request ID.

Problems:

- High memory usage
- Expensive queries
- Slow dashboards
- Monitoring system overload

Good practice:

- Do not use `user_id`, `request_id`, or `email` as metric labels.
- Use bounded labels like `service`, `route`, `status_code`, `region`.
- Put high-detail identifiers in logs and traces instead.

4. What happens if the rate limiter store fails?

There are two common strategies:

Strategy	Meaning	Trade-Off
Fail open	Allow requests if limiter fails	Better availability, weaker protection
Fail closed	Reject requests if limiter fails	Stronger protection, worse availability

Choice depends on the system.

For login or payment fraud protection, fail closed may be safer. For public read APIs, fail open may preserve availability.

5. How do you secure service-to-service communication?

Use multiple layers:

- Network isolation
- Mutual TLS where appropriate
- Service identity
- Short-lived credentials
- Least privilege access
- Authorization between services
- Audit logs
- Secrets stored in secret managers

Do not assume internal traffic is automatically trusted.

8. Comparison Tables

Logs Vs Metrics Vs Traces

Aspect	Logs	Metrics	Traces
Main purpose	Detailed event debugging	System health over time	Request path across services
Data type	Events/text/JSON	Numeric time series	Spans linked by trace ID
Best for	Root cause details	Dashboards and alerts	Distributed latency debugging
Example	Payment timeout log	p95 latency = 800 ms	Payment span took 1.2 sec
Cost at scale	Can be high	Usually lower	Medium to high
Query style	Search/filter	Aggregate/graph	Follow one request

Logs Vs Audit Logs

Aspect	Application Logs	Audit Logs
Purpose	Debugging and operations	Security and compliance
Records	Errors, events, internal state	Who did what and when
Mutability	May have shorter retention	Should be tamper-resistant
Example	Payment API timeout	Admin changed user role

Alert Vs Dashboard

Aspect	Alert	Dashboard
Purpose	Wake or notify someone	Visualize system health
Used when	Immediate action may be needed	Investigation or monitoring
Risk	Too many alerts cause fatigue	Too many dashboards cause confusion
Example	Error rate > 5%	Graph of latency by service

Fixed Window Vs Sliding Window Vs Token Bucket Vs Leaky Bucket

Algorithm	How It Works	Pros	Cons	Best Use
Fixed window	Count requests per fixed interval	Simple and fast	Boundary bursts	Basic API limits
Sliding window log	Store request timestamps	Accurate	Memory-heavy	Strict limits
Sliding window counter	Approximate with current and previous windows	Balanced	Less exact	Large-scale APIs
Token bucket	Refill tokens over time	Allows bursts	Needs token state	Public APIs

Algorithm	How It Works	Pros	Cons	Best Use
Leaky bucket	Process at steady rate	Smooth output	May queue/drop bursts	Traffic shaping

Authentication Vs Authorization

Aspect	Authentication	Authorization
Question answered	Who are you?	What can you access?
Example	Login with password	Can view this order
Failure status	Usually 401	Usually 403
Common tools	Passwords, OTP, OAuth	Roles, permissions, policies

Fail Open Vs Fail Closed

Aspect	Fail Open	Fail Closed
Meaning	Allow when control system fails	Deny when control system fails
Availability	Higher	Lower
Security	Weaker	Stronger
Example	Allow read requests if limiter fails	Block login if fraud check fails

9. Common Mistakes

- Logging sensitive information such as passwords, tokens, OTPs, or full card numbers.
- Using only average latency instead of p95 or p99.
- Creating alerts for every internal error instead of user impact.
- Not adding request IDs or trace IDs.
- Keeping rate-limit counters only in local memory when multiple servers exist.
- Forgetting to set TTL on rate-limit keys.
- Confusing authentication with authorization.
- Trusting internal services without authorization.
- Storing secrets in source code or config files committed to Git.
- Treating observability as something to add after production issues.
- Using user ID or request ID as metric labels, causing high-cardinality metrics.
- Logging too much, making useful logs hard to find and expensive to store.
- Not monitoring dependency latency, such as database, cache, queue, and external API calls.

10. Edge Cases / Special Cases

Sampling

For very high traffic systems, logging or tracing every request can be too expensive. Sampling stores only a percentage of events.

Important detail: always keep error traces or important business failure logs even if normal traffic is sampled.

Clock Skew

In distributed systems, server clocks may differ. This can make logs and traces appear out of order.

Use synchronized clocks and trace systems that handle timing carefully.

Partial Outages

A system can be healthy overall but broken for one region, tenant, route, or dependency.

Use labels such as:

```
region
service
route
status_code
dependency
```

Cascading Failure

One slow service can cause retries, which increase traffic, which makes the service slower.

Mitigations:

- Timeouts
- Retries with backoff
- Circuit breakers
- Rate limiting
- Bulkheads

Alert Flapping

An alert that repeatedly fires and resolves can be noisy.

Fix with:

- Minimum duration thresholds
- Hysteresis
- Better alert grouping

Distributed Rate Limiting Race Conditions

If two servers increment a counter at the same time, limits can be violated unless operations are atomic.

Use atomic Redis commands, Lua scripts, or purpose-built rate-limiting systems.

User Privacy In Logs

Some logs may contain personal data accidentally through request bodies or error messages.

Use:

- Redaction
- Masking
- Access control
- Retention limits

Security Of Internal Admin APIs

Internal does not mean safe. Admin APIs need authentication, authorization, audit logging, and network restrictions.

11. How to Explain in Interview

Operations means designing the production support around a system so it can be monitored, debugged, protected, and controlled. I would add structured logs with request IDs, metrics for latency, traffic, errors, and saturation, distributed traces for cross-service debugging, actionable alerts based on user impact, rate limiting to prevent abuse and overload, and basic security like authentication, authorization, input validation, encryption, secret management, and safe logging.

12. Quick Revision Notes

Key Definitions

Term	Definition
Logs	Detailed event records
Metrics	Numeric measurements over time
Traces	Request journey across services
Span	One operation inside a trace
Alert	Notification when a condition may require action
Rate limiting	Restricting request volume
Authentication	Verifying identity
Authorization	Verifying permission
SLI	What is measured
SLO	Reliability target
SLA	External contract

Important Points

- Logs are for detail.
- Metrics are for trends and alerts.
- Traces are for distributed request debugging.
- p95 and p99 matter more than average latency.

- Alerts should be actionable.
- Rate limiting protects against abuse and overload.
- Security must be designed into every layer.
- Never log sensitive data.
- Use correlation IDs across services.

Common Comparisons

Comparison	Key Difference
Logs vs Metrics	Events vs numeric time series
Metrics vs Traces	Aggregate health vs single request path
Authentication vs Authorization	Identity vs permission
Token bucket vs Leaky bucket	Burst-friendly vs steady output
SLO vs SLA	Internal target vs external contract
Fail open vs Fail closed	Availability-first vs security-first

Must-Remember Facts

- HTTP 429 means too many requests.
- HTTP 401 usually means unauthenticated.
- HTTP 403 usually means authenticated but not allowed.
- Use structured logs in production.
- Use trace IDs across service boundaries.
- Avoid high-cardinality labels in metrics.
- Use atomic operations for distributed rate limiting.

Interview Traps

- Do not say "we will check logs" without saying what logs contain.
- Do not say "average latency is fine" without checking percentiles.
- Do not design rate limiting without defining the key.
- Do not confuse authentication and authorization.
- Do not alert on every exception.
- Do not log secrets for debugging.

13. Practice Tasks

Task 1: Design Logging For Login API

Design logs for:

```
POST /login
```

Include:

- Successful login
- Failed login
- Rate-limited login
- Suspicious IP

Avoid logging:

- Password
- OTP
- Full token

Task 2: Pick Metrics For A URL Shortener

For a URL shortener, define metrics for:

- Redirect latency
- Redirect request count
- 404 rate
- Database latency
- Cache hit ratio
- Short URL creation errors

Explain which metrics should trigger alerts.

Task 3: Trace A Food Delivery Order

Draw a trace for:

```
User -> API Gateway -> Order Service -> Restaurant Service -> Payment Service ->
Delivery Assignment Service -> Notification Service
```

Mark which service is slow and how you would identify it.

Task 4: Implement A Simple Token Bucket In Python

```
import time

class TokenBucket:
    def __init__(self, capacity, refill_rate_per_second):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate_per_second
        self.last_refill = time.time()

    def allow_request(self):
        now = time.time()
        elapsed = now - self.last_refill
        self.tokens = min(self.capacity, self.tokens + elapsed * self.refill_rate)
```

```
        self.last_refill = now

        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False

bucket = TokenBucket(capacity=5, refill_rate_per_second=1)

for i in range(10):
    print(i, bucket.allow_request())
```

Expected learning:

- Initial burst is allowed.
- Later requests depend on token refill.
- This is local-only; distributed systems need shared state.

Task 5: Design Rate Limits For An API

Create rate limits for:

- Login
- Search
- Checkout
- Password reset
- File upload

For each, define:

- Key: IP, user ID, API key, or tenant
- Limit
- Time window
- Failure behavior

Task 6: Security Review Exercise

Given this bad design:

```
The API accepts user_id from the client and returns all orders for that user_id.
Logs include full request body.
JWT secret is stored in source code.
Passwords are stored using plain SHA-256.
```

Find the problems and fix them.

Expected fixes:

- Derive user ID from authenticated token, not client input.

- Check authorization for every order.
- Redact sensitive logs.
- Store secrets in a secret manager.
- Use a strong password hashing algorithm with salt.

14. Final Cheat Sheet

Core Definition

Operations in system design means making a production system observable, reliable, controlled, and secure using logs, metrics, traces, alerts, rate limiting, and security practices.

Why It Matters

It helps engineers detect problems, debug failures, prevent overload, stop abuse, protect users, and maintain reliability after deployment.

Most Asked Questions

- What is the difference between logs, metrics, and traces?
- How do you debug high latency in microservices?
- What metrics would you monitor for this system?
- What makes a good alert?
- How does rate limiting work?
- Token bucket vs leaky bucket?
- How do you implement distributed rate limiting?
- Authentication vs authorization?
- What should not be logged?
- What security basics should every API have?

Common Comparisons

Topic	Quick Difference
Logs vs Metrics	Detailed events vs numeric trends
Metrics vs Traces	Aggregate health vs one request journey
Alert vs Dashboard	Notification vs visual investigation
Token bucket vs Leaky bucket	Allows bursts vs smooths traffic
AuthN vs AuthZ	Who you are vs what you can do
SLO vs SLA	Internal target vs customer promise

One-Line Interview Answer

For production readiness, I would add structured logs with trace IDs, metrics for latency/traffic/errors/saturation, distributed tracing, actionable SLO-based alerts, rate limiting to prevent abuse and overload, and security basics like authentication, authorization, encryption, input validation, secret management, and safe logging.